

Giorno 15 — Advanced mappings: One-to-One e One-to-Many

Esercizio 1 — Relazione bidirezionale One-to-Many

Modella una relazione realistica One-to-Many: un corso di laurea ha molti studenti. Lavorerai con fetch types, lazy loading e JOIN FETCH per ottimizzare le query.

Cosa fare:

- Crea un'entità CorsoLaurea con: id, nome (String, unico), dipartimento (String), durata_anni (int). Annota con @Entity.
- Aggiungi in CorsoLaurea una lista @OneToMany(mappedBy="corsoLaurea", cascade=CascadeType.ALL) List<Studente> studenti. Aggiungi in Studente @ManyToOne @JoinColumn(name="corso_laurea_id") CorsoLaurea corsoLaurea.
- Nel DataLoader, crea 3 corsi di laurea e assegna ogni studente a un corso. Verifica nella H2 Console che la colonna CORSO_LAUREA_ID sia presente in STUDENTI.
- Crea un endpoint GET /api/corsi che restituisce la lista di tutti i corsi di laurea. Verifica cosa succede con la serializzazione JSON: potresti incorrere in riferimenti circolari. Usa @JsonManagedReference / @JsonBackReference o @JsonIgnore per risolverli.
- Sperimenta con il fetch type: aggiungi FetchType.EAGER alla relazione in CorsoLaurea e osserva le query SQL in console (show-sql=true). Poi cambia a FetchType.LAZY e confronta.
- Crea un endpoint GET /api/corsi/{id}/studenti che restituisce tutti gli studenti di un corso. Con LAZY loading, questo potrebbe generare il problema N+1. Risolvi usando @Query con JOIN FETCH nel repository.
- Aggiungi in CorsoLaurea un metodo che conti gli studenti iscritti. Crea una query JPQL che restituisca il numero di studenti per ogni corso: SELECT c.nome, COUNT(s) FROM CorsoLaurea c JOIN c.studenti s GROUP BY c.nome.

Consegna: Lancia l'applicazione con show-sql=true e confronta il numero di query generate con LAZY vs JOIN FETCH. Copia le due versioni di SQL in un file confronto-query.txt con un commento esplicativo indicando il numero ricavato.

Esercizio 2 — Esami e relazione unidirezionale

Modella la relazione tra uno Studente e i suoi Esami: uno studente può sostenere molti esami. Questa è una relazione One-to-Many unidirezionale (solo Studente conosce i suoi Esami).

Cosa fare:

- Crea un'entità Esame con: id, materia (String), voto (int, tra 18 e 30), data_esame (LocalDate), lode (boolean).
- In Studente aggiungi @OneToMany(cascade=CascadeType.ALL) @JoinColumn(name="studente_id") List<Esame> esami. Nota: non c'è mappedBy perché è unidirezionale. Verifica che la tabella ESAMI abbia la colonna STUDENTE_ID.
- Nel DataLoader, aggiungi almeno 3 esami per il primo studente e 2 per il secondo. Un esame con lode deve avere voto=30.
- Crea un endpoint GET /api/studenti/{id}/esami che restituisce la lista degli esami dello studente, ordinati per data.
- Crea un endpoint POST /api/studenti/{id}/esami che aggiunge un nuovo esame a uno studente esistente. Valida che il voto sia tra 18 e 30 (usa @Valid e @Min/@Max sull'entità, dopo aver aggiunto spring-boot-starter-validation).
- Crea un endpoint GET /api/studenti/{id}/media che calcola e restituisce la media voti dello studente come JSON: {"studente": "Mario Rossi", "media": 27.5, "totaleEsami": 4}.
- Implementa la cancellazione a cascata: elimina uno studente e verifica che i suoi esami vengano eliminati automaticamente.

► **Consegna:** L'API deve gestire correttamente il caso in cui si tenti di aggiungere un esame con voto non valido (es. 15): deve rispondere con 400 Bad Request e un messaggio di errore esplicativo.